

UNIVERSIDAD DEL VALLE DE GUATEMALA
REDES- SECCIÓN 10

Catedrático: Kevin Antonio Velázquez Aguilar



Informe Proyecto Redes y Conclusiones

Excelencia que trasciende

23661 – Mérida Jiménez, Ángel de Jesús

Nueva Guatemala de la Asunción, 21 de septiembre de 2026

Informe Proyecto Redes:

Metodología

- Herramienta: Wireshark 4.6.8 con Npcap
- Interfaz: Wi-Fi
- Cliente: 192.168.31.122 (laptop, red local)
- Servidor: ph-0a7e10cd21644e58b2b1b1b08f4907ad.ecs.us-east-2.on.aws
- IPs del servidor: 18.119.145.162, 3.19.89.14 y 77.112.76.164 (balanceador en 3 zonas)
- Archivo: docs/evidencia/captura-mcp.pcapng

Filtro de captura (aísla el tráfico hacia AWS y las consultas DNS):

```
host 77.112.76.164 or host 3.19.89.14 or host 18.119.145.162 or port 53
```

Filtro de visualización (elimina las consultas DNS de otras aplicaciones):

```
ip.addr==18.119.145.162 || ip.addr==3.19.89.14 || dns.qry.name contains "ecs.us-east-2"
```

Descifrado de TLS

El servidor en AWS solo acepta HTTPS, así que sin más configuración Wireshark mostraría registros TLS cifrados (Application Data) en lugar de los mensajes JSON-RPC.

Para descifrarlos se usó la variable de entorno SSLKEYLOGFILE. Python escribe en ese archivo los secretos de cada sesión TLS que abre, y Wireshark los lee (Preferencias → Protocols → TLS → (Pre)-Master-Secret log filename) para descifrar el tráfico capturado.

Este método no expone la clave privada del certificado del servidor: solo revela las claves efímeras de las sesiones de esta captura. Con TLS 1.3, cada sesión tiene claves propias que no sirven para descifrar ninguna otra.

Escenario capturado

1. Arranque del chatbot con MCP_REMOTE_URL apuntando a AWS
2. Pregunta: "¿tienen ibuprofeno disponible en Mixco?"
3. Cierre con /salir

Resumen del intercambio

La captura contiene seis intercambios HTTP, cada uno en su propia conexión TCP (tcp.stream 0 a 5):

- Stream 0 (frames 98 → 104): POST /mcp (180 B) → 200 OK (168 B). Mensaje initialize.
- Stream 1 (frames 121 → 126): POST /mcp (57 B) → 202 Accepted (0 B). Mensaje notifications/initialized.
- Stream 2 (frames 142 → 149): POST /mcp (51 B) → 200 OK (3372 B). Mensaje tools/list.
- Stream 3 (frames 181 → 185): POST /mcp (131 B) → 200 OK (432 B). Mensaje tools/call con search_medications.
- Stream 4 (frames 215 → 218): POST /mcp (123 B) → 200 OK (1059 B). Mensaje tools/call con check_inventory.
- Stream 5 (frames 246 → 249): DELETE /mcp (0 B) → 204 No Content. Cierre de sesión.

Los tamaños entre paréntesis son el Content-Length del cuerpo HTTP.

Clasificación de los mensajes JSON-RPC (Sección 7)

Criterio

JSON-RPC 2.0 distingue los tipos de mensaje por sus campos, no por el transporte:

- Solicitud: lleva method e id. Espera respuesta.
- Notificación: lleva method, pero no id. No espera respuesta.
- Respuesta: lleva el id de la solicitud y un result (o un error).

Sobre esa base, cada mensaje se clasifica además según su función en el protocolo MCP: los de sincronización establecen o terminan la sesión, y los de operación hacen el trabajo útil.

Mensajes de sincronización

Establecen el acuerdo entre cliente y servidor antes de cualquier operación, y lo terminan al final.

Stream 0 — initialize (solicitud, id 1)

```
{"jsonrpc": "2.0", "id": 1, "method": "initialize",  
  "params": {"protocolVersion": "2025-06-18", "capabilities": {},  
    "clientInfo": {"name": "pharmacy-chatbot", "version": "0.1.0"}}
```

El cliente declara la versión del protocolo que habla y se identifica.

Stream 0 — respuesta a initialize (respuesta, id 1)

```
{"jsonrpc": "2.0", "id": 1,
  "result": {"protocolVersion": "2025-06-18", "capabilities": {"tools": {}},
    "serverInfo": {"name": "custom-mcp-server", "version": "0.1.0"}}
```

El servidor confirma la versión y declara que ofrece herramientas ("tools": {}). En los encabezados HTTP de esta respuesta viaja además el identificador de sesión:

Mcp-Session-Id: 930546f37d0a46de8490da68252b9fb5

Stream 1 — notifications/initialized (notificación, sin id)

```
{"jsonrpc": "2.0", "method": "notifications/initialized"}
```

El cliente confirma que el handshake terminó. Al ser una notificación, el protocolo prohíbe responderla: el servidor contesta 202 Accepted con Content-Length: 0, es decir, acusa recibo a nivel HTTP pero no envía ningún mensaje JSON-RPC.

Stream 5 — cierre de sesión

```
DELETE /mcp HTTP/1.1
Mcp-Session-Id: 930546f37d0a46de8490da68252b9fb5
```

No es un mensaje JSON-RPC sino una operación del transporte HTTP de MCP: el cliente le pide al servidor liberar la sesión. La respuesta 204 No Content confirma el cierre sin cuerpo.

Mensajes de solicitud (operación)

Stream 2 — tools/list (solicitud, id 2)

```
{"jsonrpc": "2.0", "id": 2, "method": "tools/list"}
```

El cliente pregunta qué herramientas ofrece el servidor.

Stream 3 — tools/call (solicitud, id 3)

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call",
  "params": {"name": "search_medications", "arguments": {"query": "ibuprofeno"}}
```

Stream 4 — tools/call (solicitud, id 4)

```
{"jsonrpc": "2.0", "id": 4, "method": "tools/call",
  "params": {"name": "check_inventory", "arguments": {"sku": "MED-002"}}
```

Estas dos solicitudes las decidió el LLM: primero buscó el producto para obtener su SKU, y con ese SKU consultó el inventario.

Mensajes de respuesta

Stream 2 — respuesta a tools/list (id 2): las cinco herramientas con sus descripciones y JSON Schemas (3372 bytes).

Stream 3 — respuesta a tools/call (id 3):

```
{"jsonrpc": "2.0", "id": 4, "method": "tools/call",  
  "params": {"name": "check_inventory", "arguments": {"sku": "MED-002"}}}
```

Stream 4 — respuesta a tools/call (id 4):

disponibilidad de MED-002 en las tres sucursales (1059 bytes).

Nótese que el resultado de las herramientas viaja como JSON dentro de JSON: el campo text contiene una cadena con JSON escapado (\", \n). Así lo define MCP: el contenido de una herramienta es texto, y el servidor de farmacia serializa sus datos estructurados dentro de ese texto.

Resumen de la clasificación

Sincronización

- initialize (id 1): solicitud
- resultado de initialize (id 1): respuesta
- notifications/initialized: notificación
- DELETE /mcp: operación HTTP, no es JSON-RPC

Peticiones

- tools/list (id 2): solicitud
- tools/call con search_medications (id 3): solicitud
- tools/call con check_inventory (id 4): solicitud

Respuestas

- lista de herramientas (id 2)
- resultado de la búsqueda (id 3)
- resultado del inventario (id 4)

Los id numeran las solicitudes de forma secuencial (1, 2, 3, 4) y cada respuesta repite el id de su solicitud: así el cliente sabe a qué pregunta corresponde cada respuesta. La notificación no lleva id porque no tiene respuesta que emparejar.

Análisis por capas (Sección 9)

El frame 80 (consulta DNS) muestra en un solo paquete las cuatro capas apiladas; los frames 83–104 (stream 0) muestran la conexión completa del primer mensaje MCP.

Encapsulamiento

JSON-RPC 2.0 {"method": "initialize" ...}	Aplicación
HTTP/1.1 POST /mcp	Aplicación
TLS 1.3 registros cifrados	Aplicación / presentación
TCP puerto 63872 → 443	Transporte
IPv4 192.168.31.122 → 18.119.145.162	Red
Ethernet II Intel 70:36:e0 → Xiaomi	Enlace

Cada capa envuelve a la de arriba agregando su propio encabezado, y ninguna necesita conocer el contenido de las demás.

Capa de enlace

- Protocolo: Ethernet II
- MAC origen: f4:26:79:70:36:e0 (Intel, la tarjeta Wi-Fi de la laptop)
- MAC destino: 44:f7:70:03:0c:03 (XiaomiMobile, el router)

Dos observaciones:

- La MAC de destino es la del router, no la de AWS. Las direcciones MAC solo tienen alcance local: sirven para llegar al siguiente salto. En cada salto de la ruta hacia Ohio el encabezado de enlace se reemplaza, mientras que el encabezado IP se mantiene.
- La interfaz es Wi-Fi (802.11), pero Wireshark muestra tramas Ethernet II. En Windows, sin modo monitor, el controlador entrega las tramas ya convertidas al formato Ethernet, así que los encabezados propios de 802.11 no son visibles.

Capa de red

- Protocolo: IPv4
- Origen: 192.168.31.122 (dirección privada)
- Destino DNS: 192.168.31.1 (el router, que actúa como resolutor)
- Destino MCP: 18.119.145.162 y 3.19.89.14 (direcciones públicas de AWS)
- Dirección privada. 192.168.31.122 pertenece a un rango privado y no es enrutable en Internet. El router aplica NAT: reemplaza la dirección de origen por su IP pública antes de enviar el paquete. AWS nunca ve 192.168.31.122.
- Resolución DNS previa. Antes de abrir cualquier conexión, el cliente resolvió el nombre del servidor (frames 80–82). La respuesta devolvió tres direcciones, una por zona de disponibilidad del balanceador.
- Balanceo por DNS (round-robin). La primera respuesta DNS (frame 82) listaba 18.119.145.162 primero, y el cliente se conectó a esa IP. A los ~95 segundos el cliente volvió a consultar (frames 202–204), probablemente porque venció el TTL de la

caché. Esta vez la respuesta traía 3.19.89.14 primero, y los streams 4 y 5 se conectaron ahí. El balanceador reparte la carga rotando el orden de las direcciones.

Capa de transporte

Se observan los dos protocolos de transporte:

UDP (DNS, puerto 53). Sin conexión: una consulta, una respuesta, sin handshake. Adecuado para intercambios cortos donde reintentar es barato.

TCP (MCP, puerto 443). Con conexión y confiable. En el stream 0:

- Frames 83–85, handshake de 3 vías: SYN → SYN, ACK → ACK
- Frames 86–97, handshake TLS: sobre la conexión TCP ya abierta
- Frames 98–104, datos: la petición y la respuesta HTTP
- Frames 106–110, cierre: FIN, ACK de ambos lados

Opciones negociadas en el SYN (frame 83) y el SYN, ACK (frame 84):

- MSS: 1460 en ambos lados. Es el tamaño máximo de segmento.
- Window Scale: 256 en ambos lados. Multiplica la ventana anunciada.
- SACK_PERM: activado en ambos lados. Permite confirmar segmentos fuera de orden.
- Ventana inicial: 65535 el cliente y 26883 el servidor. Son los bytes que cada uno acepta sin confirmar.

Segmentación. La respuesta a tools/list mide 3372 bytes, más que el MSS de 1460. TCP la dividió en varios segmentos, y Wireshark lo indica en el frame 148: [TCP PDU reassembled in 149]. En cambio la respuesta de check_inventory (1059 bytes) cabe en un solo segmento.

Puertos efimeros. Cada conexión usa un puerto de origen distinto (63872, 63873, 63875, 63895, 63916, 53044), asignado por el sistema operativo. El puerto de destino es siempre 443.

Cierre abrupto. El último frame (253) es un RST, ACK del cliente. Tras el intercambio de FIN, Python cerró el socket y el sistema envió un RST. No afecta la comunicación, porque la respuesta ya se había recibido completa.

Capa de aplicación

Cuatro protocolos, uno sobre otro:

DNS. Traduce el nombre del servidor a direcciones IP. Consultas tipo A (IPv4).

TLS 1.3. Cifra y autentica la comunicación. En cada conexión:

- Client Hello (cliente → servidor): propone cifrados e incluye el SNI.

- Server Hello, Certificate, Finished (servidor → cliente): elige el cifrado y demuestra su identidad.
- Change Cipher Spec, Finished (cliente → servidor): confirma.
- New Session Ticket (servidor → cliente): permite reanudar la sesión más rápido.
- Alert: Close Notify (servidor → cliente): aviso de cierre ordenado (frame 250).

El SNI (Server Name Indication) viaja sin cifrar en el Client Hello: SNI=ph-0a7e10cd...ecs.us-east-2.on.aws. El balanceador lo necesita para elegir el certificado correcto antes de que exista cifrado. Consecuencia: aunque TLS protege el contenido, cualquiera que observe la red puede saber a qué servidor se conecta el cliente.

HTTP/1.1. Transporta los mensajes MCP:

- POST /mcp: envía un mensaje JSON-RPC.
- DELETE /mcp: cierra la sesión.
- 200 OK: respuesta con cuerpo JSON-RPC.
- 202 Accepted: acuse de una notificación, sin cuerpo.
- 204 No Content: confirmación del cierre de sesión.
- Mcp-Session-Id: identifica la sesión en cada petición.
- Connection: close: la conexión se cierra tras cada respuesta.
- Server: custom-mcp-server/0.1.0: identifica al servidor propio.

JSON-RPC 2.0. El protocolo de MCP, analizado en la sección 3.

Hallazgos

Una conexión nueva por cada mensaje

Es el hallazgo más relevante. Cada petición lleva el encabezado Connection: close —urllib lo agrega por defecto— así que cada mensaje JSON-RPC paga su propio handshake TCP, su propio handshake TLS y su propio cierre.

El stream 1 (la notificación) lo muestra con claridad: 21 paquetes para transportar un mensaje JSON de 57 bytes.

En tiempo, midiendo el stream 0:

- RTT (ida y vuelta): ~98 ms, del SYN (frame 83) al SYN, ACK (frame 84).
- Handshake TLS: ~98 ms, del Client Hello (frame 86) al Server Hello (frame 93).
- Petición y respuesta: ~100 ms, del POST (frame 98) al 200 OK (frame 104).
- Total del mensaje: ~303 ms, del frame 83 al 104.

Cada mensaje cuesta unos tres viajes de ida y vuelta a Ohio, y solo uno de ellos hace trabajo útil. Con conexiones persistentes (HTTP keep-alive) el cliente pagaría los handshakes una sola vez y cada mensaje siguiente costaría un solo viaje: aproximadamente un tercio del tiempo. Es la mejora más clara que deja el análisis.

La regla de las notificaciones se ve en el cable

notifications/initialized recibe 202 Accepted con cuerpo vacío. La regla de JSON-RPC de no responder notificaciones se traduce, en el transporte HTTP, en un código de estado distinto de 200 y sin mensaje JSON-RPC.

El cliente nunca habla directamente con el contenedor

Al probar el contenedor localmente, el servidor respondía HTTP/1.0 (la versión por defecto de http.server en Python). En la captura las respuestas llegan como HTTP/1.1. La diferencia se explica porque el balanceador de carga de AWS termina la conexión TLS del cliente y abre una conexión propia hacia el contenedor dentro de la red de AWS. Hay dos conexiones TCP independientes, y la captura solo ve la primera.

El SNI expone el destino

Como se describió en 4.4, el nombre del servidor viaja en claro en cada Client Hello. TLS protege el qué, pero no el a quién.

Evidencia

Capturas de pantalla en docs/evidencia/:

- 01-vista-general.jpeg: captura completa con el filtro de visualización
- 02-initialize.png: Follow HTTP Stream del stream 0
- 03-notification-202. jpeg: Follow HTTP Stream del stream 1
- 04-tools-list. jpeg: Follow HTTP Stream del stream 2
- 05-tools-call-busqueda. jpeg: Follow HTTP Stream del stream 3
- 06-tools-call-inventario. jpeg: Follow HTTP Stream del stream 4
- 07-delete-sesion. jpeg: Follow HTTP Stream del stream 5
- 08-capas-frame-80. png: detalle de las cuatro capas del frame 80
- 09-flow-graph.png: Flow Graph con el filtro http; se ve el cambio de nodo del balanceador
- 10-flow-graph-stream0.png: Flow Graph de tcp.stream eq 0; de 23 paquetes, solo 2 son el mensaje MCP

Abrir la captura ya descifrada

La captura se entrega con los secretos TLS incrustados, de modo que quien la abra ve los mensajes JSON-RPC sin configurar nada:

```
editcap --inject-secrets tls,tls-keys.log captura-mcp.pcapng captura-mcp-descifrada.pcapng
```

editcap viene incluido con Wireshark.

Conclusiones

MCP es una capa delgada sobre un formato que ya existía. El protocolo no inventa un formato de mensajes: usa JSON-RPC 2.0 y le agrega un ciclo de vida (initialize, notifications/initialized, tools/list, tools/call) y unas convenciones sobre cómo describir herramientas. Implementarlo a mano mostró que la complejidad no está en el protocolo, sino en respetar sus reglas pequeñas: no responder las notificaciones, emparejar cada respuesta con su id y separar los errores de protocolo de los errores de ejecución.

Implementar el estándar con fidelidad es lo que permite la interoperabilidad. El cliente escrito para este proyecto se conectó sin modificaciones a los servidores oficiales de Filesystem y de Git, escritos por otros, en otros lenguajes. No hubo que adaptar nada para cada uno. Es la evidencia más directa de que la implementación sigue el estándar, y de por qué MCP resuelve el problema de interoperabilidad que plantea el enunciado.

El diseño en capas se validó dos veces. En la primera entrega, reemplazar la herramienta de prueba por las cinco de farmacia no obligó a tocar el protocolo. En la segunda, llevar el servidor a la nube requirió escribir un transporte HTTP nuevo, pero `mcp_server.py` no cambió en ninguna línea. Del lado del anfitrión pasó lo mismo: el cliente HTTP expone la misma interfaz que el de stdio, así que el chatbot usa el servidor remoto exactamente como el local, que es lo que pide la funcionalidad 6.

En MCP, la descripción de una herramienta es su interfaz. El LLM no recibe instrucciones de cómo encadenar herramientas; lo deduce de sus descripciones y esquemas. Sin que nadie se lo indicara, buscó primero el producto para obtener su SKU y con ese SKU consultó el inventario. También escribió el README con el servidor de Filesystem y luego lo agregó y commiteó con el de Git, cruzando dos servidores con una sola instrucción. La calidad de esas descripciones determina el comportamiento del sistema tanto como el código.

Las reglas de seguridad deben vivir en el servidor, no en el modelo. Ante un síntoma de alarma, el servidor de farmacia devuelve una derivación médica y ningún producto;

`create_order` rechaza los medicamentos con receta; el servidor de Filesystem rechaza cualquier escritura fuera de su carpeta. En los tres casos la protección no depende de que el LLM "se porte bien": el servidor simplemente no permite la acción.

Wireshark mostró lo que el código no deja ver. Desde el código, enviar un mensaje MCP es una llamada a una función. En el cable, cada mensaje abre su propia conexión TCP y su propio handshake TLS: 21 paquetes para transportar una notificación de 57 bytes, y unos 300 ms por mensaje, de los cuales solo unos 100 ms son el intercambio útil. La causa es que el cliente HTTP envía `Connection: close`. La captura también mostró el balanceo de carga por DNS entre dos nodos, el nombre del servidor viajando sin cifrar en el SNI, y el balanceador actuando como intermediario entre el cliente y el contenedor.

Dificultades

Proveedor del LLM. El enunciado sugiere los modelos de Anthropic por sus créditos gratuitos, pero no estuvieron disponibles, así que se usó el nivel gratuito de Gemini. Después, `gemini-2.5-flash` aparecía en la lista de modelos de la cuenta, pero respondía con un error 404 para llaves nuevas. Que un modelo aparezca en el catálogo no significa que la llave pueda usarlo.

Límites del servicio gratuito. El modelo respondía con errores 503 (saturación) y 429 (cuota agotada). La primera versión de los reintentos esperaba 2, 4 y 8 segundos, pero la API pedía esperar unos 25 segundos, así que se agotaban los intentos antes de tiempo. La solución fue leer el tiempo de espera que indica la propia respuesta del servidor y respetarlo.

Servidores oficiales en Windows. Node.js no estaba instalado; una vez instalado, la terminal no lo reconocía hasta abrir una nueva, porque el PATH se lee al arrancar. Además, `npx` no se puede lanzar directamente como subproceso en Windows y hay que invocarlo con `cmd /c`. Por último, el servidor oficial de Git no ofrece una herramienta para crear repositorios, así que el anfitrión inicializa el repositorio de la carpeta `demo/` al arrancar.

Plataforma de nube. El primer intento fue Google Cloud Run, que pide un prepago de USD 30 para cuentas de facturación de Guatemala. Luego AWS App Runner, que dejó de aceptar clientes nuevos el 30 de abril de 2026. Finalmente se usó AWS ECS Express Mode, que exige construir la imagen con Docker y subirla a ECR. También hubo que evitar crear claves de acceso permanentes para el usuario raíz, usando `aws login` con credenciales temporales.

Tráfico cifrado. El servidor en AWS solo acepta HTTPS, así que una captura normal muestra registros TLS en lugar de mensajes JSON-RPC. Se resolvió con `SSLKEYLOGFILE`, que hace que Python exporte las claves de cada sesión para que Wireshark pueda descifrarlas.

Lecciones aprendidas

- Diseñar pensando en el transporte que vendrá después. Separar el protocolo del transporte desde el primer día hizo que pasar de stdio a HTTP fuera agregar código, no reescribirlo.
- Un servicio remoto va a fallar de forma transitoria. El cliente debe reintentar, y cuando el servidor indica cuánto esperar, esa indicación es mejor que cualquier estimación propia.
- Medir antes de suponer. El costo de abrir una conexión por mensaje no era visible en el código; solo apareció al mirar los paquetes.
- Verificar la documentación contra la realidad. Un modelo listado pero inutilizable y un servicio de nube que ya no acepta clientes nuevos son recordatorios de que las plataformas cambian más rápido que su documentación.

Qué se mejoraría

Conexiones persistentes. Reutilizar la conexión HTTP entre mensajes eliminaría los handshakes repetidos y reduciría el tiempo por mensaje a aproximadamente un tercio.

Pedidos persistentes. El sistema de archivos de Fargate es efímero: los pedidos se pierden cuando la tarea se reinicia. Una base de datos externa resolvería el problema, y también permitiría escalar a más de una tarea.

Autenticación en el servidor remoto. Hoy el endpoint es público y cualquiera con la URL puede crear pedidos. Un token o una API key en cada petición cerraría ese acceso.

Una interfaz gráfica. El chatbot funciona en terminal; una interfaz web, que el enunciado ofrece como extra, haría la demostración más accesible.

Comentario sobre el proyecto

El proyecto conectó los temas del curso con un protocolo que está en uso hoy. El modelo OSI y el encapsulamiento suelen estudiarse con ejemplos abstractos; aquí se pudo seguir un mismo mensaje por todas las capas: el JSON-RPC que escribe el código, el HTTP que lo transporta, el TLS que lo cifra, el TCP que lo entrega y las tramas Ethernet que lo llevan al router. Ver en el frame 80 cómo 74 bytes de DNS se convierten en 116 sumando el encabezado de cada capa hizo concreto algo que antes era un diagrama. La restricción de no usar SDKs fue la parte más valiosa. FastMCP habría ocultado exactamente lo que más enseñó: las notificaciones sin respuesta, los códigos de error, el emparejamiento por id, la sesión sobre HTTP. Y el despliegue en la nube dejó una lección que no estaba en el enunciado: desplegar no es solo técnica, también son costos, restricciones por país y servicios que desaparecen, y un buen diseño tiene que poder cambiar de plataforma sin cambiar de código.